Data Structures and Memory Ownership in ParaDiS

Developer Guide to Storage, Aliases, Lifetimes, and MPI State

ParaDiS-Extended Development Notes

August 2026

Abstract

ParaDiS is a C/C++ hybrid whose production data structures are managed mainly by procedural allocation, intrusive queues, sparse pointer tables, and phase-specific MPI buffers. A C++ pointer type therefore does not, by itself, say whether a caller owns an object. This guide documents the concrete storage model: the per-rank `Home_t` root, `Param_t`, pooled `Node_t` objects and their arm arrays, implicit and materialized segment records, spatial cells, remote and mirror domains, inclusions, FMM state, and important scratch structures. It traces their allocation, reuse, invalidation, migration, and final release paths.

The guide is written against repository commit `95b2842`. It describes the implementation as it exists, including several sharp edges that extension code must respect. Source files remain authoritative after later changes.

Contents

1 The ownership model in one page

1.1 Three meanings of "owner"

ParaDiS uses the word *owner* for three different relationships. They must not be conflated.

Storage owner
The component responsible for releasing an allocation. For example, `home->nodeBlockQ` owns production `Node_t` storage, while `home->nodeKeys` only indexes those nodes.

Domain or logical owner
The MPI domain that holds the authoritative state for an entity. A native node has a local `myTag.domainID`; a ghost node is stored in local memory but its tag names a remote logical owner.

Operation owner
The rank or endpoint selected to perform a calculation or topology mutation once. Routines such as `NodeOwnsSeg()` and `DomainOwnsSeg()` choose work responsibility; they do not decide who frees memory.

Whenever this guide says *owns* without qualification, it means storage ownership.

1.2 Per-rank ownership tree

```
Home_t (one per MPI rank)
  |- owns Param_t and most rank-global arrays
  |- owns NodeBlock_t chain
  |    `- owns actual Node_t slots
  |      `- each active/recycled node owns arm arrays
  |- owns Cell_t hash entries and their integer arrays
  |    `- cell nodeQ borrows pooled Node_t objects
  |- owns RemoteDomain_t records and lookup-array storage
  |    `- lookup entries borrow pooled ghost Node_t objects
  |- owns flat Eshelby inclusion storage when enabled
  |- owns decomposition, FMM, operation-list, and scratch state
  `- temporarily owns subcycling and communication state
```

The same pooled node can be visible simultaneously through a tag lookup, a ghost traversal array, and a cell queue. These are aliases, not additional owners. Freeing through any of them would corrupt the pool.

1.3 Lifetime classes

Lifetime	Typical structures	Boundary that invalidates or releases them
Process/rank	`Home_t`, `Param_t`, node blocks, persistent cell hash entries	`ReleaseMemory()` at normal shutdown
Decomposition epoch	active `cellList`, cell/domain intersections, `RemoteDomain_t` records	`DLBfreeOld()` followed by decomposition and communication-neighbor rebuild
Outer cycle	ghost population, collision grid, some force and inclusion scratch	migration, ghost recycle, resort, and the producer's clear routine

Lifetime	Typical structures	Boundary that invalidates or releases them
Algorithm phase	segment lists/pairs, migration buffers, topology receive buffers, mirror snapshots	caller cleanup after the relevant computation or after MPI completion
Call/local scope	temporary arrays and copied value records	return path of the allocating routine

2 Source map

File	Ownership responsibility
`include/Home.h`	Per-rank root and memory-wrapper macros.
`include/Param.h`, `include/Typedefs.h`	Configuration, parser registry records, and shared value types.
`include/Node.h`, `src/Node.cc`, `src/Util.cc`	Node layout and the C++ and legacy arm-management families.
`src/QueueOps.cc`, `src/GetNewNativeNode.cc`, `src/GetNewGhostNode.cc`	Node-block allocation, pool transitions, and active lookup installation.
`include/Segment.h`, `include/SegmentPair.h`, `src/Segment.cc`	Derived segment views and C++ list ownership.
`include/Cell.h`, `src/CellTable.cc`, `src/SortNativeNodes.cc`, `src/DLBfreeOld.cc`	Persistent cell storage and decomposition-epoch views.
`include/RemoteDomain.h`, `src/InitRemoteDomains.cc`, `src/CommSendGhosts.cc`	Remote-domain metadata, ghost lookups, and message buffers.
`src/Migrate.cc`, `src/RemapInitialTags.cc`	Ownership transfer, retagging, and temporary tag maps.
`src/RemoteOps.cc`, `src/CommSendRemesh.cc`, `src/FixRemesh.cc`	Persistent operation-list storage and transferred receive-buffer ownership.
`src/FMComm.cc`, `src/SubcyclingIntegrator.cc`	FMM hierarchy and C++ cycle-scoped subcycling state.
`src/ParadisFinish.cc`	Ordered final cleanup.

For graph mutation invariants and distributed topology replay, see `docs/Topology_and_Remesh_code.tex`. For parameter registration and broadcast details, see `docs/Control_File_to_ParaDiS_Simulation_code.tex`.

3 Home_t: the per-rank root

3.1 Construction

`InitHome()` deliberately zero-initializes the root. Most pointer fields therefore begin as null and most counters begin at zero.

```
Home_t *InitHome ()
{
        Home_t *home;

/*
 *      Calloc() zeroes allocated memory, so the only initializations
 *      needed here are any variables that should NOT start at zero!
 */
        home = (Home_t *) calloc(1, sizeof(Home_t));

        return(home);
```

Initialization then attaches independently allocated subsystems to this root. The important groups in `Home_t` are:

- rank, cycle, cached domain-boundary, and timing values;
- the owned `Param_t` and rank-zero-only parser registries;
- node-block ownership, node queues, lookup arrays, and the recycled-tag heap;
- persistent cell hash buckets plus the decomposition-epoch cell list;
- remote-domain tables, communication request/status arrays, and buffers;
- the generic decomposition pointer and optional FMM hierarchy;
- topology operation storage, collision grids, tag maps, reduction buffers, material tables, and solver state.

`Home_t` is an aggregation root, not an RAII container. Its members use different allocation families and require subsystem-specific cleanup such as `FreeDecomp()`, `FMFree()`, and `FreeOpList()`.

3.2 Procedural teardown

Normal termination runs output and solver cleanup, releases MPI groups, finalizes MPI, and then calls `ReleaseMemory()`. The cleanup order is intentional:

1. `DLBfreeOld()` invalidates decomposition-epoch cell and remote aliases.
2. `FreeCellTable()` and subsystem destructors release nested allocations.
3. every pooled node's remaining arm arrays and lock are released before its raw node block is freed;
4. lookup arrays, heaps, operation storage, decomposition data, material tables, parser registries, and optional arrays are released;
5. `Param_t` is freed late because earlier destructors consult it;
6. reduction buffers and finally `Home_t` itself are freed.

The beginning of the implementation shows the alias-first ordering and the manual walk over raw node blocks:

```
void ReleaseMemory(Home_t *home)
{
        int         i;
        Node_t      *node;
        Param_t     *param;
        NodeBlock_t *nodeBlock, *thisNodeBlock;

        if (home == (Home_t *)NULL) {
            return;
        }

        param = home->param;

```

```
/*
 *      Use existing functions (if available) to clean up some of the
 *      allocated storage.
 */
        DLBfreeOld(home);
        FreeCellTable(home);
        FreeRijm();
        FreeRijmPBC();
        FreeCellCenters();
        FreeCorrectionTable();
        FMFree(home);

#ifdef ESHELBY
/*
 *      Free the array of Eshelby inclusions if it exists
 */
        if (home->totInclusionCount > 0) { free(home->eshelbyInclusions); home->
    eshelbyInclusions=0; }
#endif


#ifdef PARALLEL
/*
 *      For multi-processor runs using the X-Window display capability
 *      we'll need to clean up the mirror domain structures
 */
        if (home->mirrorDomainKeys != (MirrorDomain_t **)NULL)
        {
            for (int domID=0; domID < home->numDomains; domID++)
            {
                if ( home->mirrorDomainKeys[domID] )
                {   MirrorDomain_Free(home, domID); }
            }
        }
#endif

/*
 *      Loop through all allocated blocks of node structures.  Free
 *      any arrays associated with individual nodes then free up the
 *      blocks of node structures.
 */
        nodeBlock = home->nodeBlockQ;

        while (nodeBlock) {

            node = nodeBlock->nodes;

            for (i = 0; i < NODE_BLOCK_COUNT; i++) {

                if (node->numNbrs) {

                    FreeNodeArms(node);

                    if (node->armCoordIndex != (int *)NULL) {
                        free(node->armCoordIndex);
                        node->armCoordIndex = NULL;
                    }
                }

```

```
                DESTROY_LOCK(&node->nodeLock);

                node++;
            }

            thisNodeBlock = nodeBlock;
            nodeBlock = nodeBlock->next;

            memset(thisNodeBlock->nodes, 0, NODE_BLOCK_COUNT * sizeof(Node_t));
            free(thisNodeBlock->nodes);
            free(thisNodeBlock);
        }

        home->nodeBlockQ = 0;

        if(home->nodeKeys) {
            free(home->nodeKeys);
            home->nodeKeys = NULL;
        }

/*
 *      Free the heap used for recycling nodes that have bee deleted
 */
        if (home->recycledNodeHeapSize > 0) {
            free(home->recycledNodeHeap);
            home->recycledNodeHeap = (int *)NULL;
        }
```

Preserve this ordering when adding a long-lived subsystem. A destructor that needs a control value or lookup table must run before that dependency is released.

4 Param_t and parsing metadata

4.1 What Param_t contains

Every MPI rank owns one `Param_t`. It combines user controls, derived constants, function pointers selected during initialization, evolving global statistics, and output counters. Most fields are inline scalars or fixed arrays. The notable direct heap member is `partialDisloDensity`, allocated according to the selected Burgers-vector group count and freed before the surrounding `Param_t`.

Rank zero allocates `ctrlParamList` and `dataParamList`. Each list owns a growable `VarData_t[]` registration array. A registration's `valList`, however, is a borrowed address, normally into the live `Param_t`; it must not be freed separately.

Object/member	Storage relationship	Release rule
`home->param`	Home-owned object on every rank	Free nested `partialDisloDensity`, then free `Param_t`
`ctrlParamList`, `dataParamList`	Home-owned, task-zero registry objects	Free each `varList`, then each list object
`VarData_t::valList`	Borrowed pointer into registered storage	Never free through the registry
mobility function pointers	Non-owning executable addresses	Re-select on each rank; no deallocation

4.2 MPI broadcast caveat

Initialization broadcasts the raw bytes of `Param_t`. Pointer-bearing parser registries are therefore kept on task zero rather than broadcast. The mobility function pointers are selected after the broadcast, and dynamically sized parameter data is allocated on the receiving rank. Any new heap-backed parameter field needs an explicit serialization/allocation policy; adding a pointer to the structure without one would copy a meaningless process-local address.

5 Tag_t and network identity

`Tag_t` is a small value type containing `domainID` and `index`; it owns no memory. A tag is resolved through one of two sparse views:

- a local tag uses `home->nodeKeys[tag.index]`;
- a remote tag uses `home->remoteDomainKeys[tag.domainID]->nodeKeys[tag.index]`.

Remote resolution may legitimately return null when the rank does not currently hold that ghost. Use `GetNodeFromTag()` or `GetNodeFromIndex()` instead of duplicating unchecked table traversal.

A tag is more durable than a raw pointer across ordinary local calculations, but it is not permanent identity across ownership transfer. Migration and some initialization/compaction paths assign new tags and build temporary `TagMap_t` value records. `DistributeTagMaps()` communicates and applies those mappings to neighbor arms, then consumes the map. Code that spans migration must participate in remapping or rebuild its keys afterward.

6 Node_t: vertex state and endpoint-owned arms

6.1 Layout

```
141 class Node_t
142 {
143    public:
144       int      flags;
145
146       real8    x , y , z ;                 //  nodal position
147       real8    fX, fY, fZ;                 //  nodal force: units=Pa*b^2)
148       real8    vX, vY, vZ;                 //  nodal velocity: units=burgMag/sec
149
150
151       real8    oldx  , oldy  , oldz  ;     //  for strain increment, Moono.Rhee
152       real8    oldvX , oldvY , oldvZ ;     //  nodal velocity at previous step
153       real8    currvX, currvY, currvZ;     //  nodal velocity at beginning of the
154                                            //  current step.  Only used during
155                                            //  timestep integration while vX, vY
156                                            //  and vZ are being determined.
157
158       Tag_t    myTag;                      // tag associated with this node
159
160       int      myIndex;                    // index associated with this node (for gpu)
161
162       int      numNbrs;                    // number of neighbors
163       Tag_t   *nbrTag;                     // array of neighbor tags
164       int     *nbrIndex;                   // array of node indices if neighbors (for gpu)
165
166       real8   *armfx, *armfy, *armfz;      //  arm specific force contribution
```

```
167       real8   *burgX, *burgY, *burgZ;       //  burgers vector
168       real8   *nx   , *ny   , *nz   ;       //  glide plane
169
170       real8   *sigbLoc;                     //  sig.b on arms (numNbr*3)
171       real8   *sigbRem;                     //  sig.b on arms (numNbr*3)
172
173       int     *armCoordIndex;               //  Array of indices (1 per arm) into
174                                             //  the mirror domain's arrays of
175                                             //  coordinates (armX, armY, armZ)
176                                             //  of nodes' neighbors.  This array
177                                             //  is only present and useful while
178                                             //  task zero is downloading the data
179                                             //  from the remote domain for
180                                             //  generating output
181
182       int     *armNodeIndex;                //  Array of node/arm indices into the
183                                             //  current node list.
184
185       int      constraint;                  //  constraint =  1 : any surface node
186                                             //  constraint =  7 : Frank-Read end
187                                             //                    points, fixed
188                                             //  constraint =  9 : Jog
189                                             //  constraint = 10 : Junction
190                                             //  constraint = 20 : cross slip(default)
191                                             //                    2-nodes non-deletable
192
193
194       int   cellIdx;                        //  cell node is currently sorted into
195       int   cell2Idx;                       //  cell2 node is currently sorted into
196       int   cell2QentIdx;                   //  Index of this node's entry in the
197                                             //  home->cell2QentArray.
198
199       int   native;                         //  1 = native node, 0 = ghost node
200
201       Node_t   *next;                       //  pointer to the next node in the queue
202                                             //  (ghost or free)
203
204       Node_t   *nextInCell;                 //  used to queue node onto the current
205                                             //  containing cell
```

A `Node_t` is both a dislocation-network vertex and the storage for all data attached to its incident arms. Arrays indexed by arm number include the neighbor tag, per-endpoint force, Burgers vector, glide-plane normal, and local/remote stress contributions. Consequently:

- the persistent graph has no single canonical heap object for a segment;
- arm index i has meaning only while the node's arm arrays keep their current layout;
- topology operations must update both endpoints' arm records and preserve reciprocal neighbor tags, Burgers-vector sign conventions, and compatible glide planes;
- inserting, removing, or reallocating arms invalidates saved arm-array pointers and can shift saved arm indices.

Use the topology helpers such as `InsertArm()`, `ChangeConnection()`, `ChangeArmBurg()`, `SplitNode()`, and `MergeNode()` rather than editing parallel arrays independently.

6.2 Two active arm-allocation families

The codebase contains two distinct ownership paths:

Legacy production helpers

`AllocNodeArms()`, `ReallocNodeArms()`, and `FreeNodeArms()` manage the main simulation arrays: `nbrTag`, the Burgers, glide-plane, and force arrays, plus `sigbLoc` and `sigbRem`. A same-degree allocation reuses capacity and reinitializes entries. These helpers do not generally own `nbrIndex`, `armCoordIndex`, or `armNodeIndex`.

C++ Node_t methods

`Node_t::Allocate_Arms()`, `Free_Arms()`, and `Recycle()` manage all declared per-arm pointers. The C++ destructor calls `Recycle()`, and copy assignment deep-copies the arrays.

Production pool nodes are raw-`calloc`ed and later raw-`free`d, so their constructors and destructors do not run. The production pool must use the legacy/manual cleanup path consistently. Topology backup helpers form an additional synchronized deep-copy path and contain an in-source warning that their array lists must stay aligned with `src/Util.cc`.

Do not mix these families. In particular, calling the narrower legacy free routine after allocating all C++ auxiliary arrays can leak or strand those auxiliaries.

6.3 Logical deletion retains capacity

```
1316 /*---------------------------------------------------------------------------
1317  *
1318  *      Function:     FreeNode
1319  *      Description:  Removes a local node from various node queues,
1320  *                    recycles the tag, adds the node to the free node
1321  *                    queue and  re-initializes the node's index in the
1322  *                    <nodeKeys> array.
1323  *
1324  *                    NOTE: We don't free the arm allocations since
1325  *                    there's a good chance they'll be the same the
1326  *                    next time.  If later it is determined the arm
1327  *                    count differs, the arm arrays will be reallocated
1328  *                    at that time.
1329  *      Arguments:
1330  *          index   Index in the local <nodeKeys> array of the pointer
1331  *                  to the node to be freed.
1332  *
1333  *-------------------------------------------------------------------------*/
1334 void FreeNode(Home_t *home, int index)
1335 {
1336         RemoveNodeFromCellQ(home, home->nodeKeys[index]);
1337         RemoveNodeFromCell2Q(home, home->nodeKeys[index]);
1338         RecycleNodeTag(home, index);
1339         PushFreeNodeQ (home, home->nodeKeys[index]);
1340         home->nodeKeys[index] = (Node_t *)NULL;
1341
1342         return;
1343 }
1344
1345
1346 /*---------------------------------------------------------------------------
1347  *
1348  *      Function:     FreeNodeArms
1349  *      Description:  Release all the specified node's arm related
1350  *                    arrays, zero out the pointers, and set the arm.
1351  *                    count to zero.
1352  *      Arguments:
```

```
 *          node    Pointer to node structure in which to free
 *                  all arm related arrays.
 *
 *-------------------------------------------------------------------------*/
void FreeNodeArms(Node_t *node)
{
        if (node->numNbrs == 0) {
            return;
        }

        free(node->nbrTag);      node->nbrTag = (Tag_t *)NULL;
        free(node->burgX);       node->burgX = (real8 *)NULL;
        free(node->burgY);       node->burgY = (real8 *)NULL;
        free(node->burgZ);       node->burgZ = (real8 *)NULL;
        free(node->armfx);       node->armfx = (real8 *)NULL;
        free(node->armfy);       node->armfy = (real8 *)NULL;
        free(node->armfz);       node->armfz = (real8 *)NULL;
        free(node->nx);          node->nx = (real8 *)NULL;
        free(node->ny);          node->ny = (real8 *)NULL;
        free(node->nz);          node->nz = (real8 *)NULL;
        free(node->sigbLoc);     node->sigbLoc = (real8 *)NULL;
        free(node->sigbRem);     node->sigbRem = (real8 *)NULL;

        node->numNbrs = 0;

        return;
}
```

`FreeNode()` means "remove this logical native node and recycle its slot." It does not free the object and intentionally does not free its arm arrays. The next user of that pool slot may reuse the capacity. Creation and unpack paths must therefore initialize every scalar and array entry they rely on; zero-initialization occurred only when the block was first allocated.

7 The node pool and its non-owning views

7.1 Block allocation

When the free queue is empty, `PopFreeNodeQ()` allocates a raw `NodeBlock_t` header and a zeroed array of `NODE_BLOCK_COUNT` nodes, links the block into `home->nodeBlockQ`, initializes locks, and places all slots on the free queue.

```
 * Function    : PopFreeNodeQ
 * Description : dequeue the node at the top of the freeNodeQ and return it
 *               to the caller. If there are no free nodes, allocate a new
 *               block of nodes, queue them all on the freeNodeQ, and then
 *               pop the first one
 *
 *-------------------------------------------------------------------------*/
Node_t *PopFreeNodeQ(Home_t *home)
{
        int             i;
        NodeBlock_t     *nodeBlock;
        Node_t          *currNode, *freeNode;

        if (home->freeNodeQ == 0) {

```

```
                nodeBlock = (NodeBlock_t *) malloc(sizeof(NodeBlock_t));
                nodeBlock->next = home->nodeBlockQ;
                home->nodeBlockQ = nodeBlock;

                nodeBlock->nodes = (Node_t *)calloc(1, NODE_BLOCK_COUNT *
                                                    sizeof(Node_t));

                currNode = nodeBlock->nodes;
                home->freeNodeQ = currNode;

                for (i = 1; i < NODE_BLOCK_COUNT; i++) {
                        INIT_LOCK(&currNode->nodeLock);
                        currNode->next = currNode + 1;
                        currNode++;
                }

                currNode->next = 0;    /* last free node */
                INIT_LOCK(&currNode->nodeLock);
                home->lastFreeNode = currNode;
        }

/*
 *      Dequeue the first free node and return it to the caller
 */
        freeNode = home->freeNodeQ;
        home->freeNodeQ = freeNode->next;

        return(freeNode);
```

Although `NodeBlock_t` also has a C++ constructor using `new[]` and a destructor using `delete[]`, that is not the main runtime path. Never destroy a production pool block with the C++ path, or destroy a C++ block with `free()`.

7.2 Pool state machine

raw block slot on `freeNodeQ` ⟶ install as native in `nodeKeys` or as ghost in remote `nodeKeys` ⟶ use through lookup/cell/ghost views ⟶ detach and recycle ⟶ same address may represent a different tag

Native creation obtains the lowest recycled tag index when possible; otherwise it advances the key high-water mark and grows `nodeKeys`. Deletion places the tag index into a min-heap, nulls the authoritative local key entry, and returns the node to the free queue.

The address of a pool slot is stable until final teardown, but its *identity is not*. A cached `Node_t*` can silently become a different node after recycle and reuse. Pointer stability is therefore not object-lifetime stability.

7.3 Views into the pool

View	Meaning	Validity rule
`home->nodeKeys`	Sparse authoritative native lookup by local tag index	Entries can be null; the pointer array may move when grown

View	Meaning	Validity rule
`RemoteDomain_t::nodeKeys`	Sparse lookup of currently materialized ghosts from one domain	Replaced during ghost communication and destroyed on decomposition rebuild
`freeNodeQ`, `ghostNodeQ`, `nativeNodeQ`	Intrusive queue heads using `Node_t::next`	A node can occupy only one of these queues; `nativeNodeQ` is a compatibility/startup view, not the steady-state authoritative collection
`ghostNodeList`	Dense pointer view for threaded ghost traversal	Only entries below `ghostNodeCount` are active; array storage may move on growth
`Cell_t::nodeQ`	Intrusive spatial membership using `nextInCell`	Rebuilt by sorting and invalid after cell reset/rebalance

`PushGhostNodeQ()` updates both the intrusive queue and dense pointer array. Its source contract requires one writer and forbids concurrent ghost traversal during update. Removed ghosts are not necessarily unlinked from the whole queue immediately, so the queue alone is not a reliable list of active ghosts.

7.4 Ghost batch recycling

```
/*-----------------------------------------------------------------------------
 *
 *      Function:       RecycleGhostNodes
 *      Description:    Put all the nodes in the ghostNodeQ back onto
 *                      the freeNodeQ and reset the ghostNodeQ to empty
 *
 *-----------------------------------------------------------------------------*/
void RecycleGhostNodes(Home_t *home)
{
        if (home->ghostNodeQ == NULL) return;  /* nothing to do */

        if (home->freeNodeQ == NULL) {
                home->freeNodeQ = home->ghostNodeQ;
        } else {
                home->lastFreeNode->next = home->ghostNodeQ;
        }

        home->lastFreeNode = home->lastGhostNode;
        home->lastGhostNode = NULL;
        home->ghostNodeQ = NULL;

        home->ghostNodeCount = 0;
        return;
}
```

Batch recycling splices ghost slots onto the free queue and resets only the active ghost count and queue heads. It does not free arms or clear all old pointer values. Consumers must honor counts and phase boundaries, not merely test whether an old pointer value is non-null.

8 Segments are usually derived views

The persistent network edge is represented twice, once in each endpoint's arm arrays. `Segment_t` materializes a convenient record containing borrowed `Node_t*` endpoints and copied positions, force values, Burgers vector, glide plane, cell, and native flags. Its destructor destroys its lock when applicable; it never deletes either endpoint.

`SegmentPair_t` borrows two `Segment_t*` records and owns no nested memory. Thus the common hierarchy is:

> caller owns segment-array storage → each `Segment_t` borrows pooled nodes → a pair list owns or borrows pair-array storage → each pair borrows segment records

The allocator for the outer list determines its release rule. The C++ `Segment_List_Create()` and `Segment_Append()` family grows arrays with `newSegment_t[]` and releases replaced arrays with `delete[]`; callers must ultimately `delete[]` the returned list. Some force kernels instead build C-allocated scratch segment arrays and free them before returning. Follow the producing API rather than assuming all `Segment_t*` pointers have the same allocator.

No segment view may outlive its endpoint population. Rebuild segment and pair lists after topology, migration, or ghost refresh.

9 Cells, collision grids, and inclusions

9.1 Persistent Cell_t storage

```
class Cell_t
{
    public:
        int     cellID;                     // cell ID encoded from xyz indices to a single int

        int     inUse;                      // toggle indicating if the associated cell struct
                                            // is in use for a specific timestep

        Cell_t *next;                       // next cell in the hash table list

        Node_t *nodeQ;                      // queue head of nodes currently in this cell
        int     nodeCount;                  // number of nodes on nodeQ

        int     inclusionQ;                 // index of first inclusions in the inclusion queue for this cell
        int     inclusionCount;             // number of inclusions in this cell

        int    *nbrList;                    // list of neighbor cell encoded indices
        int     nbrCount;                   // number of neighbor cells

        int    *domains;                    // domains that intersect cell (encoded indices)
        int     domCount;                   // number of intersecting domains

        int     baseIdx;                    // encoded index of corresp' base cell (-1 if not periodic)

        real8   xShift, yShift, zShift;     // if periodic, amount to shift corresponding base coordinates

```

```
        int     xIndex, yIndex, zIndex;     // indices of the cell in each dimension.
                                            // Note that these indices are in the range 0 <=
    xIndex <= nXcells
                                            // and likewise for Y and Z to account for the ghost
     cells

        double  center[3];                  // Defines the center position of the cell.
```

`home->cellTable` is a hash table that owns all allocated `Cell_t` records. `AddCellToTable()` uses `calloc()` and links a new cell into a bucket. Cells intentionally remain in the hash table for the whole run to avoid repeated structural allocation.

A cell owns its `nbrList` and `domains` integer arrays. It does not own the nodes reachable through `nodeQ`; those are pooled nodes linked by `nextInCell`. With Eshelby support, `inclusionQ` is an integer index into the flat Home-owned inclusion array rather than a pointer.

On load rebalance, `DLBfreeOld()` retains cell structs and static neighbor lists but clears active flags and queues, frees dynamic domain lists and `cellList`, and releases remote-domain metadata. The new decomposition rebuilds these views. At final shutdown, `FreeCellTable()` releases cell-owned arrays, locks, and cell records.

9.2 Collision cell2

The collision overlay is Home-owned scratch. `home->cell2` stores queue heads, and `cell2QentArray` stores entries containing borrowed node pointers plus integer next links. Sorting rebuilds it for collision use; `FreeNode()` removes nodes from this view before recycling them. Neither array owns a node.

9.3 Eshelby inclusion storage

`EInclusion_t` is a pointer-free value record. The Home-owned inclusion array, `eshelbyInclusions`, is contiguous and reallocatable. Locally owned inclusions come first and ghost inclusions follow them. Migration and ghost exchange can compact or reallocate the array, so code must not retain an `EInclusion_t*` across those phases. Cell queues use integer indices and are rebuilt after ownership changes.

`segpartIntersectList` is a growable, phase-scoped list keyed by copied segment tags and inclusion indices. Its producer serializes appends with the associated lock and clears/frees the list after use. Saved pointers into the list are invalid after a growth operation.

10 MPI domains, ghosts, and migration

10.1 RemoteDomain_t ownership

```
struct _remotedomain {
        int     domainIdx;      /* encoded index of this domain */
        int     numExpCells;    /* number of native cells exported to */
                                /* this domain                        */
        int     *expCells;      /* list of encoded indices of the */
                                /* exported cells                 */

        int     maxTagIndex;    /* error if tag.index exceeds this value */
        Node_t  **nodeKeys;     /* indexed by node's tag.index, points */
                                /* to Node_t                           */
```

```
26
27          int     inBufLen;
28          char    *inBuf;
29          int     outBufLen;
30          char    *outBuf;
31  };
```

`home->remoteDomainKeys` owns a domain-ID-indexed table of remote-domain records, while `home->remoteDomains` is a dense list of active IDs with primary neighbors before secondary-only neighbors. Each remote record owns:

- its `expCells` array when present;
- the storage for its sparse `nodeKeys` pointer table, but not the pooled ghost nodes referenced by its entries;
- `inBuf` and `outBuf` only for the protocol phase that allocated or received them.

Persistent Home-level MPI request and status arrays are allocated when remote domains are initialized. Protocol-specific buffers must remain allocated until the matching nonblocking requests complete.

The fields `inBufLen` and `outBufLen` are protocol-polymorphic: several ghost, velocity, and remesh paths treat them as bytes, while secondary-ghost phases use element counts. Interpret a length only within the protocol that set it.

10.2 End-of-cycle ordering

```
525     // Send any nodes that have moved beyond the domain's
526     // boundaries to the domain the node now belongs to.
527
528     Migrate(home);
529
530     // Recycle all the ghost nodes: move them back to the free Queue
531
532     RecycleGhostNodes(home);
533
534     // Sort the native nodes and Eshelby inclusions into their proper
535     // subcells.
536
537     SortNativeNodes(home);
538
539     // Communicate ghost cells to/from neighbor domains
540
541     CommSendGhosts(home);
```

This sequence is a lifetime contract:

1. `Migrate()` transfers natives that crossed a domain boundary.
2. `RecycleGhostNodes()` makes old ghost pool slots reusable.
3. `SortNativeNodes()` clears and rebuilds cell membership from the authoritative native key table.
4. `CommSendGhosts()` replaces remote lookup tables and materializes the new ghost population.

Between recycle and replacement, old remote lookup storage or old `ghostNodeList` slots may still contain pointer values to reusable pool slots. They are not valid ghosts. Do not resolve remote tags or traverse stale entries in that interval.

10.3 Migration transfers logical ownership, not storage blocks

Migration performs a value transfer:

1. the source selects outgoing native nodes and serializes their scalar and arm data into caller-owned send buffers;
2. after packing a node, the source calls `FreeNode()`, making its local pool slot and tag index reusable while retaining arm capacity;
3. the destination obtains a slot from its own local pool, assigns a new local tag, allocates or reuses arms, and unpacks values;
4. the destination records `oldTag->newTag`;
5. `DistributeTagMaps()` repairs neighbor tags across ranks and frees the temporary mapping storage;
6. send/receive arrays are released only after communication completes.

No `Node_t` allocation crosses MPI. The source and destination each own their own storage, and only serialized values cross the wire.

10.4 Operation ownership for cross-domain segments

`DomainOwnsSeg()` gives same-domain segments to that domain. For a cross-domain segment it alternates the responsible rank by even/odd cycle; remesh responsibility is the inverse of collision/separation responsibility. This creates deterministic, balanced mutation authority. It has no bearing on node-block, arm-array, or communication-buffer deallocation.

10.5 Topology operation buffers

`home->opListBuf` is a persistent, growable byte buffer. Initialization allocates an initial capacity, `ClearOpList()` resets used length while retaining capacity, and `FreeOpList()` releases it at shutdown. Operation records are packed value snapshots, not owners of live pointers.

The remesh communication path has a special alias: one send-buffer entry can refer directly to `home->opListBuf`, while other entries are allocated copies. The communication cleanup must not free the aliased persistent entry. Receive-buffer ownership is transferred to `FixRemesh()`, which consumes and frees it.

10.6 Mirror domains

Only task zero creates `MirrorDomain_t` snapshots for plotting/output. A mirror owns its key-array storage, remote-arm coordinate arrays, and optional copied inclusion array. Its node-key entries point to nodes obtained from the same common pool; `MirrorDomain_Free()` releases mirror-owned arrays and returns those nodes to the free queue. Mirror nodes are output snapshots, not primary ghosts, and their pointers are invalid after mirror cleanup.

11 Other owned subsystems

11.1 Initialization staging: InData_t

`InData_t` exists only during startup. It owns a block-read node array and those nodes' arm arrays, compatibility Burgers arrays for older inputs, and the input decomposition. Its `param` field borrows the live `Param_t`. `FreeInitArrays()` deep-frees the staging allocations, then initialization frees the

container. The steady-state `home->decomp` is independently constructed/broadcast and is not an alias of the freed staging decomposition.

11.2 Generic decomposition pointer

`home->decomp` is an owned `void*` whose concrete layout depends on the selected decomposition type. Recursive sectioning owns nested boundary arrays; recursive bisection owns a tree. Always release it through `FreeDecomp(home,home->decomp)`. A plain `free()` cannot traverse either representation correctly.

11.3 FMM hierarchy

When enabled, Home owns a `FMLayer_t[]` hierarchy. Each layer owns communication-domain arrays, a cell-ID list, and hash entries. Each `FMCell_t` owns domain-intersection and multipole/expansion coefficient arrays; its next/previous fields are only hash links. Initial setup creates the static hierarchy, while load-balance refreshes recompute dynamic intersection data. Treat `FMInit()` and `FMFree()` as the constructor/destructor boundary rather than freeing nested fields from force callers.

11.4 Subcycling state

`Subcycling_t` is deliberately opaque outside `src/SubcyclingIntegrator.cc`. It is a true C++ owner of STL maps and vectors and is created with `new` by `StartSubcycling()` and destroyed with `delete` by `FinishSubcycling()`. Its contract limits it to one outer cycle and ends it before topology or migration can invalidate its node indices, tag-pair keys, and borrowed node observations.

11.5 Material and solver state

Home embeds `BurgInfo_t`, whose dynamically generated Burgers-vector, plane, indexing, and splinterability arrays are released individually during shutdown. Optional anisotropic, spectral, ARKode, Eshelby-FMM, and related subsystems likewise have feature-specific cleanup. New conditional storage should have symmetric initialization and release under the same preprocessor guard.

12 Allocation and release matrix

Storage	Allocator/creator	Storage owner	Release or reuse path
`Home_t`	`calloc` in `InitHome()`	main simulation driver	`ReleaseMemory()` then `free(home)`
`Param_t`	`calloc` during `Initialize()`	`Home_t`	free nested density array, then `free(param)`
parser registries	`calloc`; `BindVar()` grows `varList`	`Home_t`, task zero	free `varList` and registry, never `valList` targets
node-block header and slots	`malloc` header plus `calloc` slots in `PopFreeNodeQ()`	`home->nodeBlockQ`	manual per-slot arm/lock cleanup, then `free` slots and header
runtime node arms	`AllocNodeArms()` / `ReallocNodeArms()`	owning `Node_t` slot	normally retained on `FreeNode()`; `FreeNodeArms()` or shutdown

Storage	Allocator/creator	Storage owner	Release or reuse path
native key array and recycled heap	`realloc`, heap helpers	`Home_t`	final cleanup; entries never own nodes
ghost traversal array	`realloc` in `PushGhostNodeQ()`	`Home_t`	capacity retained across cycles, freed at shutdown
`Cell_t` records	`calloc` in `AddCellToTable()`	cell hash table	`FreeCellTable()`
cell dynamic arrays	cell/decomposition initialization	owning cell or Home epoch state	`DLBfreeOld()` as appropriate, final cell cleanup
`RemoteDomain_t` and arrays	remote-domain initialization	`remoteDomainKeys` table	`DLBfreeOld()`; never free ghost nodes through key entries
protocol buffers	per communication routine	allocating protocol or transferred consumer	after matching MPI completion and unpack/consume
operation-list buffer	`InitOpList()`, grow on pack	`Home_t`	clear retains capacity; `FreeOpList()` releases
segment list from C++ list API	`newSegment_t[]`	caller	`delete[]` after all segment pairs are done
mirror domain arrays	mirror receive/unpack	task-zero mirror table	`MirrorDomain_Free()` and surrounding table cleanup
`InData_t` staging arrays	restart/input readers	`InData_t`	`FreeInitArrays()`, then free container
decomposition	type-specific allocators	`Home_t`	`FreeDecomp()`
FMM layers/cells/coefficients	`FMInit()` and helpers	`Home_t` / nested FMM records	`FMFree()`
subcycling state	`newSubcycling_t`	`Home_t` for one cycle	`delete` in `FinishSubcycling()`
inclusion array	input, ghost, and migration paths using `realloc`	`Home_t`	final free; indices rebuilt after movement

13 Pointer and index invalidation rules

Saved value	Invalidated by	Safe replacement
`Node_t*`	node removal/recycle, ghost refresh, migration, mirror cleanup	save a tag only within a stable tag epoch and re-resolve; rebuild after migration
pointer to `nodeKeys` entry/storage	key-array growth via `realloc`, DLB rebuild	re-read `home->nodeKeys` and bounds
remote-node pointer or key-table entry	ghost recycle/replacement, DLB	resolve only during a valid ghost epoch
arm-array pointer or arm index	insert/delete/reallocation of arms	locate the neighbor arm again by tag
cell queue traversal state	sorting, movement, DLB, node deletion	restart from rebuilt cell queues
`Segment_t*` or `SegmentPair_t*`	segment-list growth/destruction, topology, endpoint recycle	rebuild views and pairs

Saved value	Invalidated by	Safe replacement
`EInclusion_t*`	inclusion-array growth or compaction	store stable semantic ID where appropriate, then locate again; rebuild cell indices
pointer into tag map, op list, or intersection list	any append that grows the backing array, clear/free	use copied value/offset only under the API's documented phase
MPI send/receive buffer	completion cleanup or ownership transfer	do not access after the consumer frees it; never move/free before request completion
subcycling cache/index	topology, tag changes, node-key capacity/layout changes	keep state inside the prescribed cycle and rebuild next cycle

14 Memory API and diagnostics

Translation units including `include/Home.h` replace C `malloc()`, `calloc()`, `realloc()`, and `free()` calls with `ParadisMalloc()`, `ParadisCalloc()`, `ParadisRealloc()`, and `ParadisFree()`, including source location metadata.

```
/*
 *      Add wrappers around the memory managment functions.  If
 *      memory debugging is enabled, these wrapper functions will
 *      allow us to do some tracking of allocated memory and some
 *      checks to help identify memory corruption and memory leaks.
 */
#define free(a)       ParadisFree   (__FILE__, __func__, __LINE__, (a))
#define malloc(a)     ParadisMalloc (__FILE__, __func__, __LINE__, (a))
#define calloc(a,b)   ParadisCalloc (__FILE__, __func__, __LINE__, (a), (b))
#define realloc(a,b)  ParadisRealloc(__FILE__, __func__, __LINE__, (a), (b))

void  ParadisFree   (const char *fname, const char *func, const int ln, void   *ptr );
void *ParadisMalloc (const char *fname, const char *func, const int ln, const size_t  size);
void *ParadisCalloc (const char *fname, const char *func, const int ln, const size_t  numElem,
    const size_t size);
void *ParadisRealloc(const char *fname, const char *func, const int ln, void   *ptr, const
    size_t size);

#ifdef DEBUG_MEM
void ParadisMemCheck         (void);
void ParadisMemCheckInit     (void);
void ParadisMemCheckTerminate(void);
```

Without `DEBUG_MEM`, these wrappers delegate to the system allocator and fail fast on allocation errors. With `DEBUG_MEM`, they track blocks and guard regions; the timestep driver calls `ParadisMemCheck()` at selected phase boundaries, and termination reports/cleans the tracking layer after `ParadisFinish()`.

C++ `new/delete` do not pass through these macros. Pair allocation families exactly:

- `malloc/calloc/realloc` with `free`;
- `newT` with `delete`;
- `newT[n]` with `delete[]`;
- subsystem creators with their documented subsystem destructor.

15 Current implementation sharp edges

The following are implementation observations, not recommended ownership patterns. They matter when modifying nearby code.

1. `GetNewNativeNode()` resets only selected fields and does not release retained arm capacity. Current topology callers explicitly clear arms and set native state. Every new caller must fully initialize its slot.
2. All free, ghost, and compatibility native queues share `Node_t::next`. A node cannot be linked in two of them at once. `PushFreeNodeQ()` does not update `lastFreeNode` when pushing into an empty queue, while ghost-batch splicing relies on that tail; changes to pool transition ordering should audit this invariant.
3. Primary remote-domain records are allocated with `malloc()` and only a subset of fields is initialized immediately. Existing control flow uses count fields before pointer fields. New users should not assume every pointer is null merely because the Home root was zeroed.
4. Not every communication cleanup path nulls freed `RemoteDomain_t::inBuf/outBuf` fields or resets lengths. The values are valid only inside the protocol's documented phase, even if a stale non-null bit pattern remains afterward.
5. Normal final cleanup assumes transient `tagMap` and subcycling state have already been consumed and that mirror-table lifecycle completed. The persistent MPI request/status arrays allocated during remote-domain initialization are not visibly released by `ReleaseMemory()` in this revision. Leak-clean early-exit work should audit these paths explicitly.
6. `ReleaseMemory()` relies on manual lists of conditional fields. When adding a pointer to `Home_t`, `Param_t`, `Node_t`, `Cell_t`, or an optional subsystem, add allocation, reset/invalidation, and normal plus early-exit cleanup together.

16 Extension checklist

Before adding or retaining a pointer, buffer, or index, answer all of the following:

1. Who owns the storage, and which exact function releases it?
2. Is the pointer an object, an intrusive link, a lookup entry, or a phase-scoped borrowed view?
3. Which event invalidates it: `realloc`, topology, node recycle, migration, ghost refresh, cell sort, DLB, or shutdown?
4. Does the allocation use the C wrappers, scalar `new`, array `new[]`, or a subsystem allocator, and is the release family identical?
5. If the field is in `Param_t`, how is it reconstructed after raw MPI broadcast?
6. If it is an MPI buffer, who owns it before and after each nonblocking request, and in what units is its length stored?
7. If it refers to a node or segment, can a copied `Tag_t` plus later resolution replace a long-lived raw pointer?
8. If an array grows, are all interior pointers and saved indices either rebuilt or prohibited?
9. Is initialization complete for a recycled/raw-allocated object rather than relying on a constructor or old zeroes?
10. Are feature guards, locks, failure paths, DLB rebuild, and final cleanup symmetric?

The safest steady-state node traversal is the sparse local key table:

```
for (int i = 0; i < home->newNodeKeyPtr; ++i) {
    Node_t *node = home->nodeKeys[i];
    if (node == NULL) continue;
```

```
    /* Use node only within the current topology/migration/ghost epoch. */
}
```

Across a potentially invalidating phase, discard the pointer and resolve again from a valid, remapped tag. Across migration itself, rebuild the relation after tag maps have been distributed.

17 Summary of invariants

- `Home_t` is the practical per-rank storage root, but nested subsystems still require specialized teardown.
- `nodeBlockQ`, not lookup tables or queues, owns production node objects.
- A node owns its active or retained arm allocations; logical deletion usually recycles rather than deallocates them.
- `nodeKeys`, ghost lists, remote key tables, cell queues, segment records, and segment pairs are views with narrower validity windows.
- Ghost memory is local even when logical ownership is remote.
- MPI operation authority and segment-computation ownership are unrelated to heap ownership.
- Cells persist, while decomposition-epoch lists and associations are cleared and rebuilt.
- Migration serializes values into independently owned destination storage, changes tags, repairs references, and invalidates prior relations.
- Raw allocation of class types bypasses constructors/destructors, so the manual allocator family and shutdown order are part of the correctness model.